TUTORIEL

Base de données vectorielle Milvus

Comprendre, installer et utiliser Milvus pour la recherche vectorielle et le RAG

IMT Atlantique | TAF Login | 2026

PLAN

Sommaire

1. Contexte

- Qu'est-ce qu'un vecteur / embedding ?
- Pourquoi une BDD vectorielle ?
- Recherche traditionnelle vs vectorielle

2. Milvus

- Présentation et architecture
- Concepts clés : collections, index, métriques

3. Pratique

- Installation (Docker)
- Operations CRUD avec PyMilvus
- Recherche vectorielle

4. RAG

- Embeddings avec OpenAI
- Pipeline RAG complet
- Cas d'usage concrets

CONTEXTE

Qu'est-ce qu'un embedding ?

Un **embedding** est une représentation numérique (vecteur) d'une donnée non structurée : texte, image, audio...

- Chaque donnée est projetée dans un espace à **N dimensions**
- Des données **sémantiquement proches** ont des vecteurs proches
- Généré par un **modèle de ML** (ex: OpenAI, Sentence-Transformers)

Exemple

```
# "Roi" et "Reine" sont proches dans l'espace vectoriel
embedding_roi   = [0.12, 0.85, 0.33, ...]  # 1536 dims
embedding_reine = [0.14, 0.82, 0.35, ...]

# "Voiture" est éloigné
embedding_voiture = [0.91, 0.10, 0.67, ...]
```

Dimension typique : 384 à 3072 selon le modèle `text-embedding-3-small` → 1536 dimensions

CONTEXTE

Pourquoi une base de données vectorielle ?

Recherche sémantique

Trouver des résultats par **sens** et non par mots-clés exacts

Performance

Recherche parmi des **milliards** de vecteurs en millisecondes grâce à l'indexation ANN

IA & LLM

Composant essentiel des pipelines **RAG** pour les modèles de langage

	BDD RELATIONNELLE (SQL)	BDD VECTORIELLE
Type de requête	Correspondance exacte (WHERE, LIKE)	Similarite semantique
Données	Structurées (tables, colonnes)	Non structurées (texte, images, audio)
Scalabilité	Verticale principalement	Horizontale (distribuée)
Cas d'usage	CRUD, transactions	Recommandation, RAG, detection d'anomalies

MILVUS

Présentation de Milvus

Milvus est une base de données vectorielle open-source, conçue pour la recherche de similarité à grande échelle.

- **Open-source** sous licence Apache 2.0
- Supporte des **milliards** de vecteurs
- Recherche de similarite en **millisecondes**
- Architecture **cloud-native** et distribuée
- Support de **filtrage hybride** (scalaire + vectoriel)
- Multiple types de vecteurs : Float32, Float16, Binary, Sparse

Modes de déploiement

- **Milvus Lite** — en mémoire, idéal pour les tests
- **Standalone** — un seul nœud via Docker
- **Cluster** — distribué via Kubernetes
- **Zilliz Cloud** — service managé

Écosystème

- SDKs : Python, Java, Go, Node.js
- Integrations : LangChain, LlamaIndex, Haystack

MILVUS

Architecture de Milvus

SDK Clients
PyMilvus • Java • Go • Node.js • REST API

↓

Access Layer (Proxy)
Validation des requêtes, routage, équilibrage de charge

↓

Root Coord
DDL / Timestamps

Query Coord
Gestion des requetes

Data Coord
Gestion des données

Index Coord
Gestion des index

↓

Query Nodes
Recherche vectorielle

Data Nodes
Insertion / Suppression

Index Nodes
Construction des index

↓

etcd
Métadonnées

MinIO / S3
Stockage objets

Pulsar / Kafka
Log broker

MILVUS

Concepts clés

Collection

Équivalent d'une **table** en SQL. Contient des entités (lignes) avec des champs scalaires et vectoriels.

Index

Structure de données pour accélérer la recherche. Plusieurs algorithmes disponibles : **IVF_FLAT**, **HNSW**, **AUTOINDEX**...

Métriques de distance

- **L2** — Distance euclidienne
- **IP** — Produit scalaire (Inner Product)
- **COSINE** — Similarité cosinus

Recherche ANN

Approximate Nearest Neighbors : trouve les vecteurs les plus proches sans parcourir l'ensemble des données. Compromis précision / vitesse.

Filtrage hybride : Milvus permet de combiner une recherche vectorielle avec des filtres scalaires, ex: `filter="subject == 'history'"`

MILVUS

Métriques de distance

L2 (Euclidienne)

Mesure la distance géométrique entre deux points dans l'espace vectoriel. Plus la valeur est **faible**, plus les vecteurs sont proches.

```
d = sqrt(sum((a_i - b_i)²))
```

Idéal pour : images, données normalisées

IP (Inner Product)

Calcule le produit scalaire entre deux vecteurs. Plus la valeur est **élevée**, plus les vecteurs sont similaires.

```
d = sum(a_i * b_i)
```

Idéal pour : vecteurs normalisés, recommandation

COSINE

Mesure l'angle entre deux vecteurs, indépendamment de leur magnitude. Valeur entre **-1** et **1**.

```
d = (A · B) / (||A|| * ||B||)
```

Idéal pour : texte, embeddings de LLM

Comment choisir ? Utilisez **COSINE** pour du texte (insensible à la norme), **L2** pour des données déjà normalisées, et **IP** quand la magnitude des vecteurs est significative.

MILVUS

Algorithmes d'index

IVF_FLAT

Divise l'espace vectoriel en **clusters** (Inverted File). Lors d'une recherche, seuls les clusters les plus proches sont parcourus.

- **nlist** — nombre de clusters
- **nprobe** — clusters explorés à la recherche

Bon compromis rappel / vitesse. Recherche exacte dans chaque cluster.

HNSW

Graphe navigable en couches (**Hierarchical Navigable Small World**). Construit un graphe multi-niveaux pour une recherche très rapide.

- **M** — connexions par nœud
- **efConstruction** — qualité du graphe

Excellent rappel, plus gourmand en mémoire. Idéal pour la plupart des cas.

AUTOINDEX

Milvus choisit **automatiquement** l'algorithme et les paramètres optimaux en fonction des données et de la taille de la collection.

Recommandé pour débuter — aucune configuration nécessaire.

En pratique : Commencez avec **AUTOINDEX** puis passez à **HNSW** (meilleur rappel) ou **IVF_FLAT** (moins de mémoire) selon vos besoins.

PRATIQUE

Installation avec Docker

1. Fichier `compose.yml`

```yaml
# compose.yml (mode standalone)
services:
  milvus:
    image: milvusdb/milvus:v2.5.6
    container_name: milvus-standalone
    environment:
      ETCD_USE_EMBED: "true"
      ETCD_DATA_DIR: "/var/lib/milvus/etcd"
      COMMON_STORAGETYPE: local
    volumes:
      - milvus_data:/var/lib/milvus
    ports:
      - "19530:19530"
      - "9091:9091"

volumes:
  milvus_data:
```

2. Lancer Milvus

```bash
# Démarrer le conteneur
docker compose up -d

# Vérifier le statut
docker compose ps

# Vérifier la santé
curl http://localhost:9091/healthz
```

3. Installer le SDK Python

```bash
pip install pymilvus
```

Alternative : Pour des tests rapides sans Docker, utilisez `Milvus Lite` directement en Python avec `pip install "pymilvus[milvus_lite]"`

PRATIQUE

Créer une collection

```python
from pymilvus import MilvusClient

# Connexion au serveur Milvus
client = MilvusClient("http://localhost:19530")

# Créer une collection avec des vecteurs de dimension 5
client.create_collection(
    collection_name="demo_collection",
    dimension=5
)
```

Paramètres principaux

- `collection_name` — nom de la collection
- `dimension` — taille des vecteurs
- `metric_type` — type de distance (défaut: COSINE)
- `consistency_level` — niveau de cohérence

Schémas avancés

Pour plus de contrôle, définissez un schéma explicite avec `CollectionSchema` et `FieldSchema` pour spécifier les types de champs, les index et les contraintes.

PRATIQUE

Insérer des données

```
client.insert(
    collection_name="demo_collection",
    data=[
        {
            "id": 0,
            "vector": [0.1, 0.2, 0.3, 0.2, 0.1],
            "text": "AI was proposed in 1956.",
            "subject": "history"
        },
        {
            "id": 1,
            "vector": [0.3, 0.1, 0.4, 0.1, 0.5],
            "text": "Alan Turing was born in London.",
            "subject": "history"
        },
        {
            "id": 2,
            "vector": [0.9, 0.1, 0.2, 0.7, 0.4],
            "text": "Computational synthesis predicts molecular properties.",
            "subject": "biology"
        },
    ]
)
# {'insert_count': 3, 'ids': [0, 1, 2]}
```

Champs dynamiques : Par défaut, Milvus accepte des champs supplémentaires (`text` , `subject`) en plus de `id` et `vector` grace au schema dynamique.

PRATIQUE

Recherche vectorielle

```
results = client.search(
    collection_name="demo_collection",
    data=[[0.1, 0.2, 0.3, 0.2, 0.1]],   # vecteur de requete
    filter="subject == 'history'",         # filtre scalaire
    limit=2,                               # nombre de resultats
    output_fields=["text", "subject"],     # champs a retourner
)

# Resultat :
# [
#   {'id': 1, 'distance': 0.95, 'entity': {'text': 'Alan Turing...', 'subject': 'history'}},
#   {'id': 0, 'distance': 0.90, 'entity': {'text': 'AI was proposed...', 'subject': 'history'}}
# ]
```

Comment ça fonctionne

- Le vecteur de requête est comparé à tous les vecteurs de la collection
- L'index ANN accélère la recherche
- Le filtre `filter` est appliqué en parallèle

Supprimer des données

```
client.delete(
    collection_name="demo_collection",
    ids=[0, 2]
)
# {'delete_count': 2}
```

RAG

Générer des embeddings

Pour des données réelles, on utilise un modèle pour générer automatiquement les vecteurs.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small",
    base_url="https://models.inference.ai.azure.com",
    api_key="YOUR_API_KEY"
)

# Générer un embedding pour une phrase
vector = embeddings.embed_query("How is data stored in Milvus?")

print(len(vector))    # 1536
print(vector[:5])     # [0.009, -0.005, 0.006, -0.038, -0.018]
```

Modèles populaires

- `text-embedding-3-small` (OpenAI) — 1536 dims
- `all-MiniLM-L6-v2` (Sentence-Transformers) — 384 dims
- `BGE-M3` (BAAI) — 1024 dims

Points importants

- Toujours utiliser le **même modèle** pour l'indexation et la requête
- La **dimension** de la collection doit correspondre au modèle

RAG

Pipeline RAG avec Milvus

Retrieval-Augmented Generation : enrichir un LLM avec des données externes via la recherche vectorielle.

ETAPE 1 **Documents** → ETAPE 2 **Embeddings** → ETAPE 3 **Milvus** → ETAPE 4 **Contexte** → ETAPE 5 **LLM → Reponse**

Phase d'indexation

- Charger les documents (FAQ, docs, etc.)
- Découper en chunks
- Générer les embeddings
- Stocker dans Milvus

Phase de requête

- Transformer la question en embedding
- Rechercher les top-K documents dans Milvus
- Construire le contexte
- Envoyer question + contexte au LLM

RAG

RAG — Indexation des documents

```python
from glob import glob
from pymilvus import MilvusClient

# 1. Charger et découper les documents
text_lines = []
for file_path in glob("milvus_docs/en/faq/*.md"):
    with open(file_path, "r") as f:
        text_lines += f.read().split("# ")

# 2. Générer les embeddings et préparer les données
data = []
for i, line in enumerate(text_lines):
    data.append({
        "id": i,
        "vector": embeddings.embed_query(line),
        "text": line
    })

# 3. Créer la collection et insérer
milvus_client = MilvusClient(uri="http://localhost:19530")
milvus_client.create_collection(
    collection_name="rag_collection",
    dimension=1536,
    metric_type="IP",
)

milvus_client.insert(collection_name="rag_collection", data=data)
# {'insert_count': 72}
```

RAG

RAG — Requête et génération

```python
question = "How is data stored in Milvus?"

# 1. Recherche des documents pertinents
search_res = milvus_client.search(
    collection_name="rag_collection",
    data=[embeddings.embed_query(question)],
    limit=3,
    output_fields=["text"],
)

# 2. Construire le contexte a partir des resultats
context = "\n".join([res["entity"]["text"] for res in search_res[0]])

# 3. Interroger le LLM avec le contexte
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

model = ChatOpenAI(model="gpt-4.1-mini", base_url="...", api_key="...")

messages = [
    SystemMessage(content="You are a helpful assistant. Answer from the context."),
    HumanMessage(content=f"Context: {context}\n\nQuestion: {question}"),
]

response = model.invoke(messages)
print(response.content)
```

APPLICATIONS

Cas d'usage concrets

Chatbot RAG

Assistant qui répond à des questions en s'appuyant sur une base documentaire interne.

Recommandation

Suggérer des produits, articles ou contenus similaires à ceux déjà consultés.

Recherche d'images

Trouver des images similaires à partir d'une image requête (reverse image search).

Détection d'anomalies

Identifier des comportements inhabituels en comparant des embeddings de transactions.

Déduplication

Détecter les doublons dans de grands volumes de textes ou de données.

Et bien plus...

Drug discovery, analyse de code source, clustering, matching de CV...

CONCLUSION

Récapitulatif et ressources

Ce qu'on a vu

- Les embeddings et la recherche vectorielle
- L'architecture et les concepts de Milvus
- Les opérations CRUD avec PyMilvus
- La recherche avec filtrage hybride
- Un pipeline RAG complet

Pour aller plus loin

- Tester différents index (HNSW, IVF_FLAT, DiskANN)
- Explorer le filtrage avancé et les expressions booléennes
- Déployer en mode cluster avec Kubernetes
- Intégrer avec LangChain ou LlamaIndex

Ressources

- Documentation officielle
- Guide PyMilvus
- GitHub Milvus
- AI Quick Reference

Code du tutoriel

Retrouvez les notebooks Jupyter et le `compose.yml` sur le repository GitHub du projet.

`simpleDemo.ipynb` — Operations CRUD

`RAGDemo.ipynb` — Pipeline RAG complet

FIN

Merci !

Des questions ?